

SONGO

Version 3 — Multijoueur PHP + MySQL

Jeu de semailles des peuples Ekang — Cameroun · Gabon · Guinée équatoriale

Documentation technique

Fonctionnement · Principe · Algorithme · Organisation du code

Structure des fichiers de cette version

songo.html (structure HTML) | songo.css (mise en forme) | songo.js (logique client + polling AJAX) | api.php (routeur + handlers) | game.php (logique du jeu) | config.php (connexion MySQL) | songo.sql (schéma base de données)

1. Fonctionnement du jeu

1.1 Présentation

Le Songo est un jeu de stratégie à deux joueurs de la famille des jeux de semailles (awélé). Il est pratiqué par les peuples Efang d'Afrique centrale (Eton, Bulu, Ewondo, Béti — Cameroun, Gabon, Guinée équatoriale). Le but est de capturer plus de 40 graines sur les 70 en jeu, avant son adversaire.

1.2 Mise en place

Le plateau comporte 2 rangées de 7 cases, soit 14 cases au total. Chaque case contient 5 graines au départ (70 graines au total). Le Joueur Sud joue la rangée du bas, le Joueur Nord la rangée du haut.

Case	N1	N2	N3	N4	N5	N6	N7
Nord (rangée 0)	5	5	5	5	5	5	5
Sud (rangée 1)	5	5	5	5	5	5	5

1.3 Déroulement d'un tour

Le joueur actif choisit une case de son camp (cases surlignées en bleu). Il ramasse toutes les graines de cette case et les distribue une à une en boucle :

- Son propre camp : de droite à gauche (case 7 → case 1).
- Camp adverse : de gauche à droite (case 1 → case 7).
- Si la case contient plus de 13 graines : tour complet, la case de départ est sautée.

1.4 Récoltes (prises)

Les prises s'effectuent uniquement dans le camp adverse. La dernière graine distribuée doit tomber dans une case adverse contenant 1 à 3 graines (total 2 à 4 après dépôt) → toutes capturées.

- Prise à la chaîne : on remonte les cases précédentes contenant 2 à 4 graines, et on les capture aussi.
- Case n°1 adverse : prise d'une seule graine uniquement si au moins 14 graines ont été distribuées.

1.5 Règle de solidarité

Si le camp adverse est vide : le joueur doit obligatoirement jouer un coup qui distribue au moins 7 graines dans ce camp. Si aucun coup ne le permet, il distribue le maximum possible. Si aucun coup n'atteint le camp adverse, la partie s'arrête.

1.6 Interdits

- Interdit de semer 1 ou 2 graines chez l'adversaire en passant par sa case 7.
- Interdit de vider complètement le camp adverse lors d'une prise (aucune capture effectuée si c'est le cas).

1.7 Fin de partie

Condition	Résolution
Un joueur atteint 40 graines capturées	Ce joueur gagne immédiatement
Moins de 10 graines sur le plateau	Chaque joueur récupère les graines de son camp ; le plus riche gagne
Solidarité impossible (camp adverse vide)	Idem — chaque joueur récupère son camp
Égalité (aucun n'atteint 40)	Partie nulle

2. Principe

2.1 Nature du jeu

Le Songo est un jeu combinatoire parfait à information complète : les deux joueurs voient l'intégralité du plateau à tout moment. Il n'y a aucun élément de hasard. La victoire repose uniquement sur la planification et l'anticipation du point de chute de la dernière graine distribuée.

2.2 Tension stratégique

- Capturer un maximum de graines dans le camp adverse.
- Éviter de laisser des cases vulnérables (2 à 3 graines) dans son propre camp.
- Gérer la solidarité : parfois sacrifier une capture pour alimenter l'adversaire.

2.3 Architecture client-serveur

Dans cette version, deux joueurs se connectent depuis des machines distinctes via un navigateur web. L'état de la partie réside entièrement sur le serveur PHP/MySQL. Le frontend (songo.js) interroge l'API toutes les 2 secondes (polling AJAX) pour détecter les changements et mettre à jour l'affichage.

Principe du polling AJAX

Joueur A et Joueur B interrogent indépendamment le serveur toutes les 2 secondes via GET ?action=state. Le serveur retourne l'état complet de la partie (plateau, scores, cases jouables calculées selon le token du demandeur). Dès que Joueur A joue (POST ?action=move), Joueur B verra le changement au prochain poll (délai max 2s).

3. Algorithme

3.1 Représentation des données (MySQL)

L'état de chaque partie est stocké dans une seule ligne de la table games en base MySQL. Le plateau et les scores sont sérialisés en JSON dans des colonnes TEXT.

```
-- Table principale
CREATE TABLE games (
  id          VARCHAR(36) PRIMARY KEY,      -- UUID de la partie
  code        VARCHAR(6)  UNIQUE,           -- Code à partager (ex:
AB12CD)
  board       TEXT,                          -- JSON:
[[5,5,5,5,5,5,5],[5,5,5,5,5,5]]
  scores      TEXT,                          -- JSON: [scoreNord, scoreSud]
  current_player TINYINT,                    -- 1=Sud, 2=Nord
  status      ENUM('waiting','playing','finished'),
  token_sud   VARCHAR(36),                  -- UUID joueur Sud
  token_nord  VARCHAR(36),                  -- UUID joueur Nord
  log         TEXT,                         -- JSON: tableau de messages
  highlight   TEXT,                         -- JSON: {last, captured}
  last_activity DATETIME
);
```

3.2 Navigation sur le plateau (nextPos)

La fonction nextPos(\$row, \$col, \$player) calcule la case suivante dans le sens de distribution. Le circuit est asymétrique selon le joueur : Sud tourne dans un sens, Nord dans l'autre.

```
function nextPos(int $row, int $col, int $player): array {
  if ($player === 1) { // Sud : row1 col- → row0 col+
    if ($row === 1) return $col > 0 ? [1, $col-1] : [0, 0];
    else           return $col < 6 ? [0, $col+1] : [1, 6];
  } else { // Nord : row0 col+ → row1 col-
    if ($row === 0) return $col < 6 ? [0, $col+1] : [1, 6];
    else           return $col > 0 ? [1, $col-1] : [0, 0];
  }
}
```

3.3 Distribution des graines (applyMove)

```
$seeds = $board[$own][$col];
$board[$own][$col] = 0;
$row = $own; $c = $col; $s = $seeds;
while ($s > 0) {
  [$row, $c] = nextPos($row, $c, $player);
  // Sauter la case de départ si grand tour (>13 graines)
  if ($row=== $own && $c=== $col && $seeds>13) { $s--; continue; }
  $board[$row][$c]++;
  $s--;
}
// ($row, $c) = position de la dernière graine distribuée
```

3.4 Calcul des prises

```
if ($pr === $opp) {
    $fc = oppFirstCol($player); // case n°1 adverse
    if ($pc === $fc && $seeds >= 14) {
        // Cas spécial case n°1 : 1 seule graine capturée
        capturer 1 graine;
    } else {
        // Prise normale + chaîne
        while ($pr === $opp && $pc !== $fc) {
            $cnt = $board[$pr][$pc];
            if ($cnt >= 2 && $cnt <= 4) {
                capturer $cnt; $board[$pr][$pc] = 0;
                [$pr, $pc] = prevPos($pr, $pc, $player);
            } else break;
        }
    }
}
```

3.5 Vérification des interdits

```
// Interdit : vider complètement le camp adverse
$wouldEmpty = count(array_filter($board[$opp], fn($x) => $x > 0)) === 0;
if ($wouldEmpty && count($captureList) > 0) {
    foreach ($captureList as [$r, $c, $cnt])
        $board[$r][$c] += $cnt; // annuler toutes les prises
    $captureList = [];
}
```

3.6 Cases jouables — solidarité

```
function getSelectableCells(array $board, int $player): array {
    // Cas normal : toutes les cases non vides du joueur
    if (!$oppEmpty) return $allOwn;

    // Solidarité : coups qui distribuent >=7 graines chez l'adversaire
    $reach7 = array_filter($allOwn,
        fn($c) => simulateDistInOpp($board, $c, $player) >= 7);
    if (count($reach7) > 0) return array_values($reach7);

    // Sinon : coup qui distribue le maximum
    $maxDist = max(array_map(
        fn($c) => simulateDistInOpp($board, $c, $player), $allOwn));
    return array_values(array_filter($allOwn,
        fn($c) => simulateDistInOpp($board, $c, $player) === $maxDist));
}
```

3.7 Endpoints de l'API REST

Méthode + Action	Corps / Params	Réponse
POST ?action=create	— aucun corps	gameId, code, token, playerNum=1
POST ?action=join	{ code }	gameId, code, token, playerNum=2

GET ?action=state	?gameId=&token=	État complet de la partie
POST ?action=move	{ gameId, token, col }	État mis à jour après le coup

4. Organisation du code

4.1 Structure des fichiers

Le projet est organisé en 7 fichiers aux responsabilités distinctes. Le frontend (HTML/CSS/JS) est séparé du backend PHP. La logique du jeu est isolée dans game.php, indépendante de la couche HTTP.

```
songo/
├── songo.html      ← Structure HTML de l'interface (lobby + plateau)
├── songo.css       ← Mise en forme : plateau, cases, lobby, statuts
├── songo.js        ← Logique client : polling AJAX, rendu UI, actions
├── api.php         ← Routeur HTTP (4 actions) + handlers
├── game.php        ← Logique pure du jeu (navigation, prises, règles)
├── config.php      ← Connexion PDO MySQL (host, user, password, db)
└── songo.sql       ← Schéma de la base de données
```

4.2 Rôle détaillé de chaque fichier

Fichier	Rôle	Contenu clé
songo.html	Structure de la page	Div #lobby, #game, #board-wrap, #log-wrap — aucun style inline
songo.css	Toute la mise en forme	Variables CSS, .cell, .sel, .ldrop, .capt, .pinfo, #status-msg
songo.js	Client AJAX + rendu	createGame(), joinGame(), startPoll(), handle(), renderBoard(), sendMove()
api.php	Routeur + handlers	switch action → createGame(), joinGame(), getState(), handleMove()
game.php	Moteur du jeu	nextPos(), prevPos(), applyMove(), checkEnd(), getSelectableCells(), gameStateFor()
config.php	Base de données	getDB() → PDO singleton, constantes DB_HOST/DB_NAME/DB_USER/DB_PASS
songo.sql	Schéma MySQL	CREATE TABLE games avec tous les champs + index

4.3 Flux d'exécution complet

```
// Joueur A — créer une partie
[songo.js] createGame()
  → POST api.php?action=create
  ← { gameId, code:"AB12CD", token, playerNum:1 }
  → startPoll() : GET api.php?action=state toutes les 2s

// Joueur B — rejoindre
[songo.js] joinGame(code)
  → POST api.php?action=join { code:"AB12CD" }
  ← { gameId, token, playerNum:2 }
```



```

→ showGame() + startPoll()

// Joueur A clique une case
[songo.js] sendMove(col)
→ POST api.php?action=move { gameId, token, col }
[api.php] handleMove()
    [game.php] applyMove() → distribution + prises + log
    [game.php] checkEnd() → fin de partie ?
    UPDATE games SET board=..., scores=..., highlight=... WHERE id=...
← état mis à jour → handle(state) → renderBoard() + renderScores()

// Joueur B voit le coup au prochain poll (≤2s)
[songo.js] poll() → GET api.php?action=state
← nouvel état (highlight du dernier coup visible)
→ handle(state) → rendu mis à jour

```

4.4 Séparation des responsabilités

Couche	Ce qu'elle fait	Ce qu'elle NE fait pas
songo.html	Déclare la structure DOM	Aucune logique, aucun style inline
songo.css	Tout le visuel et les états (.sel, .ldrop...)	Aucune logique JS
songo.js	Polling, rendu UI, envoi des coups	Ne valide JAMAIS les règles du jeu
api.php	Authentifie le token, route l'action	Ne connaît pas les règles du jeu
game.php	Valide et applique les règles du jeu	Ne touche jamais HTTP ni le DOM
config.php	Fournit la connexion PDO (singleton)	Ne contient aucune logique métier

4.5 Sécurité des coups

Chaque requête POST ?action=move est validée côté serveur par api.php avant d'appeler game.php :

- Le token correspond à un joueur valide de la partie (token_sud ou token_nord).
- C'est bien le tour de ce joueur (current_player === playerNum).
- La colonne choisie est dans la liste retournée par getSelectableCells().
- La ligne est verrouillée (FOR UPDATE) pour éviter les doubles coups simultanés.

4.6 Installation sur XAMPP (Linux)

```

# 1. Démarrer XAMPP
sudo /opt/lampp/lampp start

# 2. Copier les fichiers dans htdocs
sudo cp -r songo/ /opt/lampp/htdocs/

# 3. Créer la base de données dans phpMyAdmin
# → http://localhost/phpmyadmin
# → Onglet SQL → coller songo.sql → Exécuter

```

```
# 4. Ouvrir le jeu
#   → http://localhost/songo/songo.html

# 5. Partager via Ngrok
ngrok http 80
# → https://abc123.ngrok.io
# Changer l'URL dans songo.js :
# let API = 'https://abc123.ngrok.io/songo/api.php'
```

Récapitulatif — Version 3 PHP

7 fichiers aux responsabilités strictement séparées : HTML (structure), CSS (mise en forme), JS (client AJAX), api.php (routeur HTTP), game.php (moteur du jeu), config.php (base de données), songo.sql (schéma). La logique du jeu est entièrement côté serveur. Le client ne valide rien. Polling AJAX toutes les 2 secondes. Tokens UUID pour l'authentification. MySQL pour la persistance. Verrou FOR UPDATE pour la cohérence des coups simultanés.